

# The payment button never unlocks

A form-compaction option added to the payment form on August 19 violates a Stripe integration rule and throws an uncaught exception. The "Processing..." button stays locked forever across all five payment flows.

● FIXED · IN PRODUCTION SINCE AUG 20

|                                                                                              |                                                                                |                                                          |
|----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|----------------------------------------------------------|
| <div>SEVERITY</div> <div>Critical — no payment could complete while the fault was live</div> | <div>STARTED</div> <div>2026-08-19 10:16</div>                                 | <div>ROOT CAUSE FOUND</div> <div>2026-08-20 ~00:50</div> |
| <div>DURATION</div> <div>≈ 15 h, resolved in code</div>                                      | <div>SCOPE</div> <div>5 flows: shop, card, event, rental, session resume</div> | <div>TRIGGERING COMMIT</div> <div>917b3db</div>          |

## Triggering cause

Commit 917b3db (Aug 19, 10:16 — cycle §66, "Stripe checkout compacted") adds four options to the `PaymentElement` in the shared component `StripePaymentStep.tsx`. One of them is the direct cause of the fault:

|                                                                                                                                                                                              |                                 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|
| THE LINE THAT INTRODUCED THE FAULT                                                                                                                                                           | 917b3db · StripePaymentStep.tsx |
| <pre>&lt;PaymentElement   options={{     layout: 'accordion',     fields: { billingDetails: { address: 'never' } },     defaultValues: customerEmail ? { ... } : undefined,   }} /&gt;</pre> |                                 |

Setting a `billingDetails` field to `'never'` tells Stripe "don't ask the user for this." Stripe allows it, but on one precise condition: the omitted data must be supplied manually at confirmation time, inside `confirmParams.payment_method_data.billing_details`. That part was never written — `confirmParams` still contains only `return_url`.

The option was added to compact the form. Its accompanying comment claimed no caller expected a billing address: true, but beside the point. The constraint isn't about what our application expects — it's about what Stripe requires.

## How a rule violation became a permanent lockup

The integration error alone would have produced an error message. What turned it into a silent, permanent lockup was the combination of four factors, all in our own code.

### 01 Stripe detects the inconsistent configuration

When `confirmPayment()` is called, missing `address.country`, Stripe rejects the confirmation.

### 02 The error is *thrown*, not returned

`IntegrationError`s are exceptions. They don't arrive in the `{ error }` object the code checks, so every existing error-handling branch is bypassed.

### 03 Nothing catches the exception

`handleConfirm` had no `try/catch`, and `onClick` ignores the returned promise. The exception leaves the function and surfaces as an unhandled rejection.

### 04 The button state never resets

Every `setIsConfirming(false)` sits further down the flow, now unreachable. The button stays locked on "Processing..." until the user leaves the page — with no message at all.

#### EVIDENCE — BROWSER CONSOLE, /CARD

Aug 20 · diagnosis confirmed

```
Uncaught (in promise) IntegrationError: You specified "never" for
fields.billing_details.address when creating the payment Element, but
did not pass confirmParams.payment_method_data.billing_details.address
.country when calling stripe.confirmPayment().
```

The `Uncaught (in promise)` prefix alone confirms points 03 and 04: nothing ever caught the exception.

# Timeline

---

Aug 18 · 23:25  
b3fc525

## Shared component is born

The four payment flows are unified into `StripePaymentStep.tsx`.  
`PaymentElement` has no options set: working correctly. From this point on, every change to this file touches all flows at once.

Aug 19 · 10:16  
917b3db

## The fault ships

Cycle §66. Four options added in one block, including `address: 'never'`. From this moment on, payment confirmation throws an exception across all five flows.

Aug 19 · 18:50  
5346f27

## Unrelated change

Dedicated Stripe key for e2e tests. No impact on the Payment Element — listed only to rule it out during the bisection.

Aug 19 · evening

## First diagnosis: Link suspected

The Link sign-up banner, which appeared at the same time, is mistaken for the cause. The explanation is plausible, and disabling Link is decided.

Aug 20 · 00:39  
f953166

## The symptom is masked

Cycle §72: payment methods restricted to `card`, plus a 28-second timeout on the button. The permanent lockup turns into a generic error after half a minute — the underlying exception stays intact and invisible.

Aug 20 · ~00:50

## Real cause identified

With the timeout removed, bisecting the component's four commits isolates §66. The console confirms the `IntegrationError`: Link has nothing to do with it.

Aug 20 · overnight

## Fix ready

Cycles §73–§75 completed and verified with `pnpm typecheck`, delivered as an archive per an explicit request not to push directly.

Aug 20 · 01:35–10:35  
c076f9c...10bbd7b

## Fix applied to `main`

The delivered archive is uploaded to `main` manually. Verified in this session via `git fetch` and a direct comparison: all twelve files from cycles §73–§75 are byte-identical to what was delivered. **The incident is closed on the code side.**

## Impact

---

- All five entry points that use the shared component: shop checkout, free-amount payment on `/card`, event booking, service rental, resuming a pending session.
- The user receives no message at all: the button stays disabled, and the only way out is to leave the page.
- The `PaymentIntent` `incomplete` on Stripe are the accounting trace of this incident: attempts interrupted by this exception, not spontaneous customer abandonment.
- Between Aug 19 10:16 and Aug 20 00:39 the lockup was permanent; after the §72 timeout it turned into a generic error after 28 seconds, still with no successful payment.

## Why it went unnoticed for so long

---

### The most visible symptom wasn't the cause

The Link banner appeared *because of* the same commit — the form, stripped of its fields, left room for Link's own interface — and was mistaken for the problem's origin. Two effects of the same cause, read as cause and effect.

### Four options introduced in a single commit

Layout, fields, prefill, and theming were added together. Bisection isolated the commit, not the line: pinning down which of the four options was responsible took an extra step.

### The email correlation misled the investigation

The fault appeared to affect only `/card`, the one module with an optional email field. In reality email prefill had nothing to do with it: the other modules had the same problem, simply not yet tested.

### The timeout hid the evidence

Added in good faith as a safety net, it replaced the lockup with a generic message — making it harder to trace back to the exception. Removing it was the step that made diagnosis possible.

### An analysis run against a stale repository state

One phase of the investigation concluded the previous fix wasn't in production, based on the local repository while the remote branch had been rewritten. Wrong conclusion, corrected after a `fetch`: deployed state must always be checked against the remote.

## Fix

| CHANGE                                                    | WHY                                                                                                                                              | STATUS  |
|-----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| Removed <code>address: 'never'</code>                     | Eliminates the cause. Reverts to the default, where Stripe itself decides which fields are actually needed and requires nothing at confirmation. | DONE    |
| <code>try/catch</code> in <code>handleConfirm</code>      | No exception, present or future, can leave the button locked without a message again.                                                            | DONE    |
| <code>try/catch</code> in all 5 <code>createIntent</code> | A dropped network connection during the call produced the same lockup, through a different path.                                                 | DONE    |
| Removed email prefill                                     | Passing a known email triggers Link authentication — the likely origin of the "too many attempts" lockouts seen in testing.                      | DONE    |
| Restored <code>automatic_payment_methods</code>           | The restriction to <code>card</code> had been introduced against Link, which turned out to be unrelated to this incident.                        | DONE    |
| Caption on the "Country" field                            | Clarifies that it means the card's country, not the customer's location.                                                                         | DONE    |
| Deployed to production                                    | Verified against <code>origin/main</code> on Aug 20: all twelve files from the three cycles are present, byte-identical to what was delivered.   | DONE    |
| Disable Link in the Stripe Dashboard                      | A decision independent of this incident: reduces friction and account-lockout risk. Not automatable from code.                                   | PENDING |

### One action still open

The incident is closed on the code side: the fix is on `main`. What remains is manually disabling Link in the Stripe Dashboard (Settings → Payment Methods → Link) — a decision made to reduce friction, independent of this incident's cause.

## What to change next time

---

- **Third-party options carry contracts, not just effects.**

Turn 'never'

off

Strip

colle

of

a

field

crea

an

oblig

to

supp

it

elsew

Before

setti

an

optio

to

or *requires in return*

equivalent,

check

what

that

value

.

- **A payment flow can't have dead-end branches.**

The `try/catch`

now

added

isn't

a

one-

off

patch

—

it's

the

minimum

requirement

for

any

function

that

disables

a

control

while

awaiting

a

promise.

- **A shared component multiplies blast radius.**

One `StripePaymentStep.tsx`

line

in

coverage `typecheck`

five

flow

Chai

to

this

file

deser

a

check

again

a

real

flow

not

just

.

– **Console before hypotheses.**

The  
error  
was  
visible  
and  
self-  
explanatory  
from  
minute  
one.  
Three  
work  
cycles  
were  
spent  
on  
hypotheses  
a  
console  
read  
would  
have  
ruled  
out  
immediately.

– **One change per commit when it touches a third-party integration.**

Had  
the  
four  
§66  
options  
been  
introduced  
separately,  
bisection  
would  
have  
been  
immediate.